

自适应分区算法的 GPU 深度优化建议

从 GPU-resident 到 register-resident，以及 Repair / Merge 从 Full Sort 到 Exact Selection

基于 partition_gpu.py 与 partition_kernels.py 的代码分析与改造方案

结论摘要 当前实现已经做到“数据和控制状态尽量留在 GPU 上、root 子树独立并行、CUDA Graph 消除大部分 host launch 开销”。下一阶段真正值得优化的方向，是进一步减少 kernel 内部对 HBM 的中间状态往返：让小树的 energy、split、DP、staircase 状态尽量驻留寄存器或被 bit-pack；同时把 repair / merge 中只为得到阈值或前 deficit 个元素而执行的全量排序，替换为保持精确语义的 radix selection / lexicographic top-k selection。两类改造可以叠加，目标是把“GPU-resident”提升为“working-set register-resident”。

1. 当前实现与本文优化目标

本文针对两份实现：高层调度 `partition\_gpu.py` 与 Triton GPU 核心 `partition\_kernels.py`。现有设计已经避免在 build 过程中读取 device scalar 回 host，并利用一棵 root 子树一个 Triton program 的方式完成 λ -DP、leaf mask、gain 和 staircase 等操作；merge 则采用 Ward cost、non-overlap matching 和 compaction。本文不改变自适应分区的基本数学目标，主要研究如何减少 GPU 上的中间内存流量、排序开销和寄存器 spill。

本文聚焦两个相互关联的方向。第一，Register-resident：尽可能把一个小 root 的工作集留在寄存器或紧凑 bit-packed 状态中，降低 global memory 的反复 load/store。第二，Exact Selection：对于只需要第 k 个阈值或前 deficit 个节点的环节，不再生成完整有序排列，而是通过 radix-select 或逐字段 lexicographic selection 直接确定需要的集合，同时保持当前 tie-break 语义。

模块	当前特点	主要问题	优先优化方向
Score tree	每个 root 独立；逐 C 通道累	每个 C 通道对 flat 做 global	C-block + 寄存器累加，最后

模块	当前特点	主要问题	优先优化方向
	计 SSE	read-modify-write; FP64 流量高	一次写回
λ -DP	一个 root $\times$ 一组 λ ; KC 最大 16	[KC,R] FP64/int 状态可能造成高寄存器压力和 spill	KC/warps autotune; 中间状态最小化
Leaf mask	bottom-up 后把 split 写 scratch, 再 top-down 读回	31 个左右 split bit 被按 byte/global tensor 保存	bit-pack 到 uint32/uint64 或保持 SSA/register
Repair	gain 排序 + staircase 多字段 stable sort	构造完整 rank/permutation, 排序和临时张量流量大	保留必要 rank; 最终 popped 用 lexicographic exact selection
Merge threshold	k-th threshold 通过 torch.sort 得到	只要 kth, 却做完整排序	FP64 ordered-key radix-select
Merge matching	整条 path 单 Triton program	大 cap 时 SM 横向并行度有限	先做 selection; 若仍是瓶颈再做分层 scan

2. 第一部分：从 GPU-resident 进一步走向 Register-resident

2.1 “GPU-resident”与“Register-resident”的区别

当前代码已经是 GPU-resident: λ bracket、leaf count、num_leaves、merge count 等关键标量不回 host, 整个 build 可被 CUDA Graph 捕获。但 GPU-resident 只说明“数据不离开 GPU”, 并不意味着数据不离开 HBM/L2。多个 kernel 或同一 kernel 的不同阶段仍可能反复把中间状态写到 global memory, 再读回来。

目标: Global-memory round trip $\downarrow \rightarrow$ Register / local tensor lifetime $\uparrow \rightarrow$ HBM bytes per root $\downarrow$

对这套算法而言, root 通常很小。例如 root=256、atom=8 时, R=32, 一棵完全二叉树只有 63 个节点, 内部节点只有 31 个。这样的 working set 很适合寄存器驻留或紧凑位图表示。真正需要警惕的是 FP64 live range 过长导致寄存器占用过高, 最终 spill 到 local memory。因此“寄存器化”应当与 KC、num_warps、C-block 一起 autotune, 而不是简单把所有数组强行展开。

2.2 优化 A : Score Tree 的 energy 由 global 累加改为寄存器累加

当前 `\_score\_tree\_kernel` 对 C 个 score 通道循环。每处理一个通道, 都会在各层计算 SSE, 并通过 `load(flat) + e -> store(flat)` 累加到 global `flat`。对于 C=128, 这意味着同一个 tree node 的 FP64

energy 被反复读写 128 次。代码位置：`partition\_kernels.py` 的 `\_score\_tree\_kernel`，尤其是逐 C 循环及各层 `tl.store(... tl.load(flat)+e)`。

```
当前逻辑（概念化）：
for c in C:
    e_level0 = SSE(channel c, atoms)
    flat[level0] = flat[level0] + e_level0
    e_level1 = reduce(e_level0 moments)
    flat[level1] = flat[level1] + e_level1
    ...
```

建议把每一级的 energy accumulator 变成 program 内 local tensor，在若干 C 通道上累积完成后再写回。最激进版本是一棵 root 的 63 个 energy 全程驻留寄存器，C 循环结束后一次写 `flat`；更稳妥的版本是按 C-block（例如 4/8/16 个通道）计算 partial energy，再合并到较小的局部 accumulator。

```
建议逻辑：
acc_l0[32] = 0; acc_l1[16] = 0; ... acc_root[1] = 0
for cb in range(0, C, C_BLOCK):
    # load C_BLOCK × 256 scores
    # 对每个 channel 计算 m / m2，并逐层生成 SSE
    acc_l0 += sum_over_channels(e0)
    acc_l1 += sum_over_channels(e1)
    ...
# 仅在最后写一次
store(flat_l0, acc_l0); store(flat_l1, acc_l1); ...
```

参数	建议起点	原因
C_BLOCK	4 / 8 / 16 三档 autotune	过大增加 FP64 live tensors 和寄存器压力；过小则无法充分减少循环/访存
num_warps	4、8 对比	root 内 R=32 不需要过多 warp；更多 warp 不一定有利于 FP64 local tensor
flat 写回	每级一次或整棵树一次	目标是从 $O(C \times n_nodes)$ global RMW 降为 $O(n_nodes)$ store
适用性	先确认实际 raw builder 是否走该 kernel	源码明确说明旧 [128,N] 路径当前可能不是主路径，避免优化非热点

预期收益判断 这一项的收益高度依赖真实路径。如果 `\_score\_tree\_kernel` 仍处于关键路径，理论上可显著减少 FP64 HBM read-modify-write；如果 raw FP8 builder 已完全绕过它，则应把同样的“局部累加后一次写回”原则应用到真正的 summary/energy builder 上。

2.3 优化 B : `dp\_leaf\_mask` 的 split scratch 改为 bit-packed register state

`\_dp\_leaf\_mask\_kernel` 先 bottom-up 计算 `do\_split`，把每个内部节点的 split decision 写入 `split\_ptr`；随后 top-down 再从 `split\_ptr` 读取，用于判断节点是否 active 以及是否输出 leaf。root=256、atom=8 时内部节点只有 31 个，本质上只需要 31 bit。当前用 uint8 scratch，并发生 global store/load。

$$31 \text{ split decisions} \approx 31 \text{ bit} < 1 \times \text{uint32}$$

建议提供两个实现版本。版本 1 最保守：仍按 level 计算，但将每一级 `ds` 打包到一个 uint32/uint64 中，最终只写极少量 packed words；top-down 阶段从 packed word 解位。版本 2 更激进：在同一个 Triton program 内直接保留每一级的 `ds` local tensor，bottom-up 完成后立即进行 top-down traceback，不落 global scratch。

```
bit-pack 方案（概念化）：
uint64 split_bits = 0
bit_offset = 0
for level in bottom_up_levels:
    ds = (split_cost < keep_cost)
    split_bits |= pack_bits(ds) << bit_offset
    bit_offset += num_nodes(level)

# top-down
active_root = 1
for level in top_down_levels:
    ds = unpack_bits(split_bits, level_offset, num_nodes)
    leaf = active & ~ds
    active_children = repeat(active & ds, 2)
```

对于 Triton，完全 register-resident 是否成立要用编译结果验证：若编译器因为跨 loop 的 heterogeneous tensor live range 而 spill，则 bit-packed uint64 方案通常更稳。它用少量整数寄存器换掉 `split\_ptr[n\_nodes]` 的读写，同时减少 scratch allocation。

2.4 优化 C : Staircase 不保存完整 `stairs[n\_nodes, LPAD]`，直接生成 packed key

当前 `\_staircase\_kernel` 同时维护 `stairs\_ptr[n\_nodes, LPAD]` 和 `keys\_ptr[n\_nodes, NKEYS]`。父节点的 staircase 会从 `stairs\_ptr` 读出，扩展后再写回，并随后把 staircase pack 成若干 int64 key。这里存在“双份状态”：完整 staircase 矩阵只是为了供子节点继承，而最终 repair 真正消费的是 packed lexicographic key。

优化方向是让“父状态”本身就采用更紧凑的表示。由于 staircase 的长度最多等于树深度（root=256、atom=8 时约 5-6 层），可以考虑以下两个层次：

- 方案 C1 - 保守：仍保留逐层 parent state，但将 `stairs` 从 int64 矩阵压缩为固定的 packed rank key，子节点通过位操作插入新的 `own\_r`，减少 scratch 宽度。

- 方案 C2 - 激进：每个 root program 在 top-down loop 中把上一层的 packed parent key 作为 local tensor 直接传给下一层，只把最终每个 node 的 `keys` 和 `reach` 写 global，不再写 `stairs\_ptr`。
 需要注意：当前 staircase 的语义不是简单把 rank 顺序 append；它会保留路径上的 record-low 序列，因此 packed 更新逻辑必须与现实完全等价。建议先在 Python reference 上实现 `packed\_staircase\_reference`，逐节点和当前 `\_staircase()` 比对，再迁移到 Triton。

2.5 优化 D : Gain 与 Repair Key 融合，避免立 FP64 gain buffer 长时间驻留

当前 repair 先通过 `\_gain\_kernel` 生成 `gain[n\_nodes]` FP64 数组，再在上层代码中用 gain + tiebreak 做全局优先级排序。若后续按照本文第二部分改为 radix/selection，可以把 gain 计算与 ordered-key 生成融合：直接输出可排序的 64-bit primary key，而不是先落一个 FP64 gain buffer、之后再读取并转换。

gain_fp64 → order-preserving uint64 key → radix / select

如果仍需保留原始 gain 做验证，可在 debug/validation 模式下保留；性能模式只输出 ordered key。这样能减少一次 FP64 array 的读写，并让后续 selection 使用整数路径。

2.6 优化 E : KC 不是越大越好 - 对 λ -DP 做寄存器压力 autotune

`\_dp\_count\_kernel` 和 `\_dp\_count\_bracket\_kernel` 会同时维护 `[KC,R]` 的 `cost` 与 `cnt`。当 KC=16、R=32 时，仅逻辑上的 FP64 cost 就有 512 个元素，再加 int32 count 和临时 split/keep；这非常容易形成较高的 register pressure。源码当前优先选择 KC=16，因此必须用真实编译结果确认是否发生 local-memory spill。

配置	可能优点	可能缺点	建议
KC=16	一次 tree load 服务 16 个 λ ； 参数并行度最高	live state 最大，可能降低 occupancy 或 spill	作为基线，不应默认最优
KC=8	仍能充分复用 tree；状态减 半	program 数增多	很可能是更平衡的候选
KC=4	寄存器压力最低， occupancy 较高	重复读取 tree 增多	适合寄存器紧张或 FP64 吞吐 弱的 GPU

测量指标不要只看 kernel time，还应同时看 `registers per thread`、`local load/store`、occupancy、L2 hit rate。若 KC=16 出现显著 local spill，则即使 launch 数更少，也可能慢于 KC=8。

2.7 Register-resident 部分的推荐落地顺序

1. 先用 Nsight Systems 确定真实热点路径，确认 score-tree、DP、staircase、merge 各自的 GPU 时间占比。
2. 对 `\_dp\_leaf\_mask\_kernel` 做 split bit-pack，这是实现风险最低且最容易做 bit-exact 对比的一项。
3. 对 λ -DP 做 KC=4/8/16、num_warps 的 autotune，并检查 local spill。
4. 如果 score-tree 仍是热点，再做 C-block + register accumulator。
5. 最后改 staircase scratch，因为它与 repair exact ordering 耦合最深。

3. 第二部分：Repair / Merge 从 Full Sort 改为 Exact Selection

3.1 先澄：不是所有 Sort 都能直接成 Top-K

Repair 中存在两类排序，其需求不同。第一类是把 `(gain desc, tiebreak asc)` 转为所有节点的 heap priority rank；当前 staircase 构造会读取每个节点的 rank，因此这里如果保持现有 staircase 定义，就需要“全体节点的相对 rank”，不能简单只保留 Top-deficit。第二类是在 staircase key 已经生成以后，代码对多个 key 字段做 stable sort，最终只用于判断 `pos < deficit`。这一类不需要完整排列，可以自然替换为精确 lexicographic selection。

排序位置	当前用途	能否直接改 selection	推荐改法
gain+tiebreak 全局 rank	给所有 node 赋 heap priority rank	不能直接只取 Top-K	先改为 fused radix rank/sort；或后续重构 repair 才能消掉全排序
staircase keys 多字段 stable sort	最终只需要前 deficit 个 reachable nodes	可以	lexicographic exact top-deficit selection
merge kth threshold	只需要第 k 小 Ward cost	可以，且非常直接	FP64 radix-select
cap_merges 二次阈值	只需要 selected edges 中第 k 小 cost	可以	同一个 radix-select primitive 复用

3.2 Merge：把 `torch.sort(cost)` 改为 FP64 Radix-Select

Merge 的 kth threshold 是最适合先改的一处。Ward cost 理论上非负，无资格的 edge 被设置为 +inf；当前只需要第 k 小值，并通过 `nextafter(kth,+inf)` 让 boundary ties 全部进入。完整排序产生 $O(N \log N)$ 的比较/搬移，而 selection 只需要确定 kth key。

对非负 FP64，IEEE-754 bit pattern 与数值大小天然单调（正数区间），因此 merge cost 可以直接 bitcast 为 uint64 key；若要复用到可能含负数的 gain，则使用通用 order-preserving transform。

```

通用 FP64 -> ordered uint64 (概念) :
bits = bitcast_u64(x)
if sign_bit(bits) == 1:
    key = ~bits          # 负数反转
else:
    key = bits ^ 0x8000000000000000 # 正数搬到高半区
# key 升序 == x 升序

```

精确 radix-select 可以按 8 bit digit 逐轮定位 kth 所在 bucket。每轮不是排序，而是统计当前候选前缀下的 256 个 histogram bins，找到累计数量首次跨过 k 的 bin，更新 key prefix 和新的 k。最多 8 轮即可精确确定 64-bit kth key。

```

RadixSelectKth(keys, valid_mask, k):
    prefix = 0; prefix_bits = 0
    for byte in [63:56, 55:48, ..., 7:0]:
        hist[256] = histogram(next_byte(key) for keys matching prefix)
        b = first bucket with cumulative_count >= k
        k -= count(buckets < b)
        prefix = prefix || b
    return exact_key(prefix)

```

为了避免后续轮次仍扫描全部 edge，可以在前 2-3 个字节后对命中 prefix 的候选 index 做一次 compaction。如果 N 只有几千到 1.6 万，也可以先实现“全量扫描 8 轮”的简单版本，验证语义和收益后再加 compaction。

Exact tie 语义 当前 merge 用 `nextafter(kth,+inf)`，等价于选取 `cost <= kth` 的 boundary ties。radix-select 得到 kth 的 exact uint64 key 后，bitcast 回 FP64，再保持原 `nextafter` 逻辑即可获得完全相同的边界语义。

3.3 Repair : 把最终 staircase 全排序改成 Lexicographic Top-deficit Selection

当前 repair 得到 packed staircase keys 后，会从最低有效字段开始做多次 stable sort，形成完整 permutation，再构造 `pos`，最后只使用 `popped = reach & (pos < deficit)`。换言之，完整 permutation 的绝大多数顺序信息最终被丢弃。

真正需要的结果：集合 S = “lexicographic order 最小的 deficit 个 reachable nodes”

因此可以直接做逐字段的 lexicographic selection，而不是逐字段 stable full sort。设 staircase 有 NKEYS 个 int64 字段 `K0, K1, ...`，排序方向均为升序。维护候选集合 `A` 和剩余名额 `k=deficit`：

```

A = reach
k = deficit
popped = false
for field f = 0 .. NKEYS-1:
    # 只在 A 中选择第 k 小的 Kf
    thr = RadixSelectKth(Kf[A], k)

```

```

strict = A & (Kf < thr)
equal  = A & (Kf == thr)
popped |= strict
k -= count(strict)
if k == 0: break
A = equal

```

```

# 完全同 key 的最终 tie: 追加 node_id 作为最后字段, 精确复现 stable sort
if k > 0:
    select k smallest node_id within A

```

这个算法是精确的, 因为 lexicographic 排序的前 k 个元素可以通过“第一字段确定边界 -> 仅在边界相等集合中看第二字段 -> 继续”得到。所有第一字段严格小于阈值的元素必然属于前 k; 只有等于阈值的元素需要进入下一字段继续决策。

当前实现以 `perm=arange` 开始并使用 stable sort, 因此如果所有 packed staircase 字段完全相同, 最终以原 node index 维持稳定次序。为了 bit-exact parity, 新的 selection 必须把 `node\_id` 作为隐式最后一个 lexicographic 字段。

3.4 Repair 初始 gain rank : 推荐先从“全比较排序”改为“整数 radix rank”, 不要强行 Top-K

在现有 staircase 定义不变的前提下, `rank[node]` 表示所有节点在 `(gain desc, tiebreak asc)` 下的完整优先级。因为每个 reachable node 的 staircase 都可能用到祖先或自身 rank, 所以这里不是只需要前 deficit 个。最稳妥的优化不是 selection, 而是把两次 `torch.sort` 融合成一次整数 radix ordering。

可以把 gain 转换成 ordered uint64 primary key, 并调整为“gain 越大 key 越小”; tiebreak 已经是 int64。然后进行 128-bit lexicographic radix sort / rank。若不希望写完整 128-bit sort, 可分两阶段稳定 radix: 先按 tiebreak 升序, 再按 gain key 升序, 直接输出 `rank`, 避免 `idx1`、`idx2`、`order` 等多份 permutation 临时张量。

```

目标顺序: gain desc, tiebreak asc
primary = ordered_u64_for_desc(gain)
secondary = tiebreak

# stable radix two-key ordering
perm = radix_sort(secondary, ascending=True)
perm = stable_radix_sort(primary[perm], ascending=True)
rank[perm] = arange(n_nodes)

```

更激进的下一阶段才是完全重构 repair, 使其不需要所有节点 rank, 例如直接在 frontier 上执行批量 exact best-first selection。但那会改变 staircase 的内部实现, 验证成本更高, 不建议作为第一版优化。

3.5 Selection primitive 的 GPU 设计建议

设计项	建议
Digit 宽度	先用 8 bit/256 bins，逻辑简单、shared/register histogram 可控；再 benchmark 10-12 bit
两级 histogram	每个 CTA 生成局部 hist；第二个小 kernel 汇总所有 CTA hist 并找到 kth bucket
候选过滤	前几轮可全量扫描；候选比例降到 1/16 或更低后做 index compaction
数据类型	Merge cost 非负 FP64 可直接用正数 bit order；Gain 使用通用 signed-float order transform
无效值	Merge 中 +inf 自然排在最后；Repair unreachable 可映射到最大 key
ties	selection 返回 exact threshold key；严格小于直接入选，相等集合继续处理下一字段或 final node_id
复用	实现一个 `radix_select_u64(keys, mask, k, order)` primitive，merge kth 与 repair lexicographic selection 共用

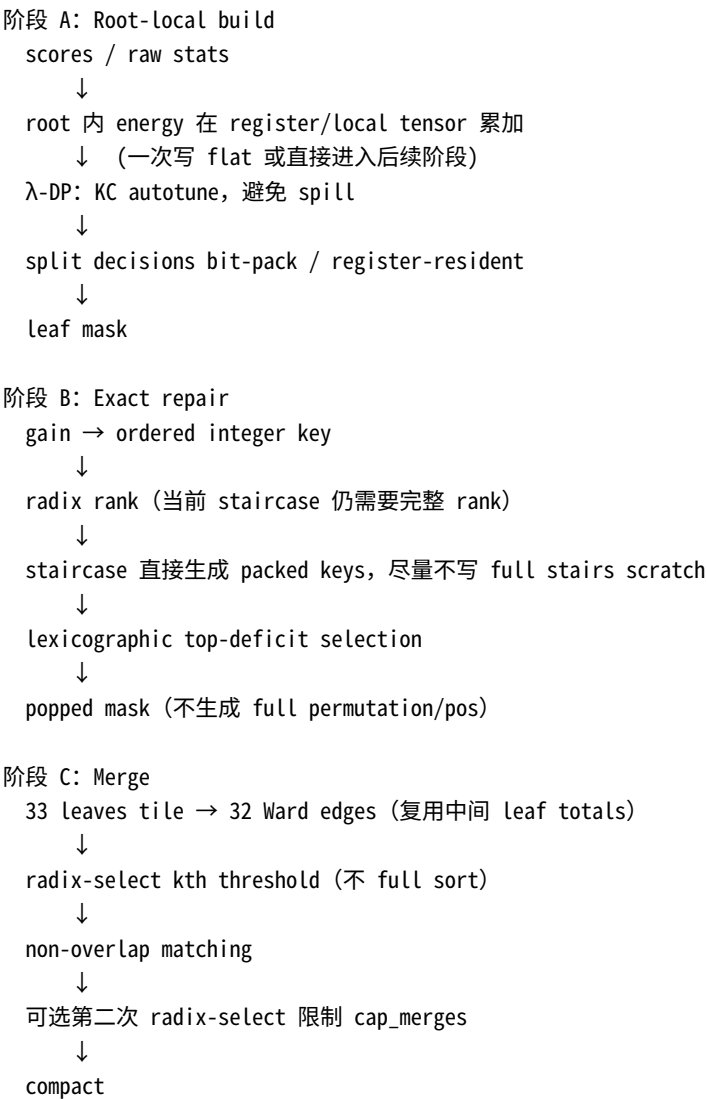
3.6 为什么 Selection 对本算法尤其合适

这套分区算法中的 sort 多数并不是为了得到一个完整有序数组，而是为了构造阈值或决定一个有限集合：merge 只需要 kth cost；repair 最终只需要前 deficit 个 staircase node。GPU 上 full sort 的主要成本不仅是比较，还包括 permutation 搬移、临时 workspace、多个 kernel 以及大规模 HBM traffic。Radix selection 将目标转为 histogram + prefix decision，整数访问规则规则、并行度高，而且与 CUDA Graph 固定 shape 兼容。

Full Sort: $O(N \log N)$ ordering + permutation traffic $\rightarrow$ Selection: $O(P \times N)$ streaming histogram

这里 P 是 radix passes。64-bit key 用 8-bit digit 时最坏 $P=8$ ，但每一轮都是连续扫描和小 histogram；若中途 compaction，后几轮只扫很小的候选集合。对 $N \approx$ 几千到 1.6 万且每层/每 merge round 重复执行的场景，这种结构值得实测。

4. 两部分组合后的推荐新执行路径



5. 预期收益、风险与验证方法

优化项	主要收益来源	语义风险	验证重点	优先级
split bit-pack	减少 scratch global R/W; 更小 workspace	低	leaf mask bit-exact	P0
KC autotune	避免 spill, 提高 occupancy	无	local load/store=0 或 显著下降	P0
score-tree register accumulate	减少 FP64 RMW	低	flat energy bit parity; 注意加法顺序	P1
merge kth radix-select	消除 full sort	低	kth、ties、+inf 语义完全一致	P1

优化项	主要收益来源	语义风险	验证重点	优先级
repair lexicographic selection	消除多轮 stable full sort/permutation	中	popped mask 与 reference 完全一致	P1
staircase packed state	减少 `stairs` scratch	中	packed key 与当前 keys 完全一致	P2
gain radix rank	减少两次 comparison sort/临时 permutation	中	rank 全量一致，尤其 tie-break	P2

5.1 必保持的正性不量

- 最终 `leaf\_start`、`leaf\_len`、`num\_leaves` 与当前 reference 逐元素一致。
- λ -DP 中 `split < keep` 的 tie 行为不变：相等时保持 parent。
- Repair 优先级仍为 gain 降序、start 升序、level 升序、idx 升序。
- Staircase lexicographic 顺序及 stable tie 行为不变；完全相同 key 时显式使用 node_id 作为最终 tie-break。
- Merge kth threshold 仍保留 boundary ties，行为与当前 `nextafter(kth,+inf)` 一致。
- Merge matching 的 non-overlap 与 `(cost,start)` greedy reference 一致。

5.2 性能测量指标

类别	建议指标
时间	每个 kernel GPU duration；tree/DP/repair/merge 分阶段总时间；整层 CUDA Graph replay 时间
寄存器	registers/thread、occupancy、active warps/SM、local load/store (spill)
内存	DRAM read/write bytes、L2 hit rate、global load/store transactions
排序/选择	sort workspace bytes、radix passes、候选 compaction 后规模、每轮 histogram 时间
系统	kernel launch 数、graph replay 时间、输入 copy/output clone 占比

6. 建议的分阶段实现路线

Phase 0 : 建立可量化基线

对 32K / 64K / 128K token，至少覆盖 C=128、典型 root/atom 配置，记录整层及每阶段 GPU 时间。用 Nsight Compute 检查 DP kernel 的寄存器数和 local spill，用 Nsight Systems 看 sort、copy、kernel launch 时间占比。只有确认热点后再进入下一阶段。

Phase 1 : 低风险高收益

6. ``dp_leaf_mask`` split bit-pack 或 local state;
7. KC=4/8/16 autotune;
8. merge kth threshold 实现 ``radix_select_u64``，替换第一处 full sort;
9. 复用同一 primitive 替换 cap_merges 的第二处 kth sort。

Phase 2 : Repair 核心优化

10. 保留现有 gain rank 语义，先把 comparison stable sort 改为 radix rank;
11. 把 staircase 最终 multi-key full sort 改为 lexicographic top-deficit selection;
12. 确认 ``popped`` mask bit-exact 后，再考虑去掉完整 stairs scratch。

Phase 3 : 进一步压缩 HBM 流量

13. 若 score-tree 是热点，做 C-block + register accumulation;
14. merge cost 改为 33-leaf / 32-edge tile reuse;
15. 如果 ``_merge_match_kernel[(1,)]`` 在大 cap 下成为瓶颈，再做 multi-program hierarchical scan，而不是过早复杂化。

7. 最终建议

对于这套自适应分区算法，GPU 优化的核心不应是把它强行改造成 GEMM，而应利用它“每棵树很小、root 之间独立、全局决策点有限”的结构。最理想的实现形态是：root-local 的树状态尽可能留在寄存器；只有 λ bracket、repair boundary、merge threshold 等真正需要全局信息的地方才落 global memory；而这些全局决策尽量使用 histogram / radix selection，而不是完整排序。

如果只选择两项最值得优先投入的工作，我建议：第一，先做 ``dp_leaf_mask`` bit-pack + KC autotune，确认能够降低 global traffic 且不引入 spill；第二，做一个通用、精确的 ``radix_select_u64``，先替换 merge 的 kth sort，再扩展为 repair 的 lexicographic top-deficit selection。后者一旦成熟，可以成为整套 adaptive partition 中通用的 GPU selection primitive。

附录 A：与当前源码的对应位置

源码模块	关键位置	本文建议
partition_kernels.py `_score_tree_kernel`	约 L173-L204	C-block、energy register accumulator、一次性 flat store
`_dp_count_kernel` / `_dp_count_bracket_kernel`	约 L35-L96	KC/num_warps autotune，控制 [KC,R] FP64 live set
`_dp_leaf_mask_kernel`	约 L127-L170	split uint32/uint64 bit-pack 或 SSA/local state
`_gain_kernel`	约 L206-L225	与 ordered key 生成融合
`_staircase_kernel`	约 L228-L277	减少 stairs scratch，直接生成 packed keys
`_merge_cost_kernel`	约 L291-L326	33-leaf/32-edge tile reuse
`merge_round_triton`	约 L499-L570	两处 kth full sort 改 radix-select
partition_gpu.py `repair_leaves`	约 L440-L483	最终 stable multi-key sort 改 lexicographic top-deficit selection
partition_gpu.py `kth_cost_threshold`	约 L574-L585	full sort -> exact kth selection

说明：本文中的“当前实现”描述基于用户提供的 partition_gpu.py 与 partition_kernels.py；“优化建议”属于在保持现有算法语义前提下的 GPU 实现推导，需要通过 bit-exact parity test 与 Nsight profiling 验证实际收益。